

Chapter 3

COMPLETING THE BASICS

Chapter 3

- 1. Assignment statement
- 2. Formatting the output
- 3. Using mathematical library functions
- 4. Program input using the *cin* object
- 5. Strings

1. Assignment statement

- **How do we place data items into variables?**
 - Read in values typed at the keyboard by the user
 - Use an assignment statement

- **Assignment statement examples:**

```
length = 25;  
cMyCar = "Mercedes";  
sum = 3 + 7;  
newtotal = 18.3*amount;
```

- *Assignment operator (=)* are used for assignment a value to a variable and for performing computations.
- Assignment statement has the syntax:
$$\textit{variable} = \textit{expression};$$
- *Expression* is any combination of constants, variables, and function calls that can be evaluated to yield a result.

Assignment statement (cont.)

- **The order of events when the computer executes an assignment statement is**
 - Evaluate the expression on the right hand side of the assignment operator.
 - Store the resultant value of the expression in the variable on the left hand side of the assignment operator.
- **Note:**
 1. The equal sign here does not have the same meaning as an equal sign in mathematics.
 2. Each time a new value is stored in a variable, the old one is overwritten.

Example 3.1.1

- **This program calculates the volume of a cylinder, given its radius and height**

```
#include <iostream>
using namespace std;
int main()
{
    float radius, height, volume;
    radius = 2.5;
    height = 16.0;
    volume = 3.1416 * radius * radius * height;
    cout << "The volume of the cylinder is " << volume << endl;
    return 0;
}
```

- **The output of the above program:**
The volume of the cylinder is 314.16

Assignment Variations

C++ includes other arithmetic operators in addition to the equal sign of assignment.

Operator	Example	Meaning
=	iNum1 = iNum2	
+=	iNum1 += iNum2	iNum1 = iNum1 + iNum2
-=	iNum1 -= iNum2	iNum1 = iNum1 - iNum2
*=	iNum1 *= iNum2	iNum1 = iNum1 * iNum2
/=	iNum1 /= iNum2	iNum1 = iNum1 / iNum2
%=	iNum1 %= iNum2	iNum1 = iNum1 % iNum2

**So `sum += 10` is equivalent to
`sum = sum + 10`**

Data Type Conversion across Assignment Operator

- **Note**: *Data type conversion* can take place across assignment operators, i.e., the value of the expression on the right side is converted to the data type of the variable to the left side.
- For example, if *temp* is an integer variable, the assignment *temp* = 25.89 causes the integer value 25 to be stored in the integer variable *temp*.

Increment and decrement operators

- For the special case in which a variable is either increased or decreased by 1, C++ provides two unary operators: *increment operator* and *decrement operator*.

Operator	Description

++	Increase an operand by a value of one
--	Decrease an operand by a value of one

- The *increment* (++) and *decrement* (--) unary operators can be used as prefix or postfix operators to increase or decrease value.

Increment and decrement operators (cont.)

- A *prefix* operator is placed *before* a variable and returns the value of the operand *after* the operation is performed.
- A *postfix* operator is placed *after* a variable and returns the value of the operand *before* the operation is performed.
- Prefix and postfix operators have different effects when used in a statement

`b = ++a; // prefix way`

will first increase the value of a (equal to 5) to 6, and then assign that new value to b. It is equivalent to

`a = a + 1; b = a;`

`b = a++;` // postfix way

will first assign the value of `a` (equal to 5) to `b`, and then increase the value of `a` to 6. It is equivalent to

`b = a; a = a + 1;`

- **The decrement operators are used in a similar way.**

`b = --a;`

equivalent to

`a = a - 1; b = a;`

`b = a--;`

equivalent to

`b = a; a = a - 1;`

■ Example 3.1.2

```
#include <iostream.h>
using namespace std;
int main()
{
    int c;
    c = 5;
    cout << c << endl;        // print 5
    cout << c++ << endl;       // print 5 then postincrement
    cout << c << endl << endl; // print 6
    c = 5;
    cout << c << endl;        // print 5
    cout << ++c << endl;
    // preincrement then print 6
    cout << c << endl;        // print 6
    return 0;
}
```

The output of the above program:

```
5
5
6

5
6
6
```

2. FORMATTING FOR PROGRAM OUTPUT

- Besides displaying correct results, a program should present its results attractively with good formats.
- **Stream Manipulators**

Stream manipulator functions are special stream functions that change certain characteristics of the input and output.

The main advantage of using manipulator functions is they facilitate the formatting of the input and output streams.

Stream Manipulators

- *setw()* The *setw()* stands for *set width*. This manipulator is used to specify the minimum number of the character positions on the output field a variable will consume.
- *setprecision()* The *setprecision()* is used to control the number of digits of an output stream display of a floating-point value. *Setprecision(2)* means 2 digits of precision to the right of the decimal point (this is only true if fixed mode is enabled).

Example:

```
cout << "|" << setw(10)  
      << setprecision(3) << fixed << 25.67 << "|";
```

cause the printout

without the "fixed" keyword, the precision starts at the beginning of the number. With the "fixed" keyword, it starts at the decimal point. To reset the behaviour after using "fixed", we can use "defaultfloat"

```
| 25.670|
```

Example 3.2.1

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    cout << setw(3) << 6 << endl
         << setw(3) << 18 << endl
         << setw(3) << 124 << endl
         << "---\n"
         << (6+18+124) << endl;
    return 0;
}
```

The output of the above program:

```
  6
 18
124
- - -
148
```

3. USING MATHEMATICAL LIBRARY FUNCTIONS

- C++ provides standard library functions that can be included in a program.
- If your program uses mathematic function *sqrt()*, it should have the preprocessor command *#include<math.h>* in the beginning of the program.

Function Name	Description	Return Value

abs(a)	Absolute value	Same data type as argument
log(a)	Natural logarithm	double
sin(a)	sine of a (a in radians)	double
cos(a)	cosine of a (a in radians)	double
tan(a)	tangent of a (a in radians)	double

MATHEMATICAL LIBRARY FUNCTIONS

Function Name	Description	Return Value

log10(a)	common log (base 10) of a	double
pow(a1,a2)	a1 raised to the a2 power	double
exp(a)	e^a $e \sim 2.71828$	double
sqrt(a)	square root of a	double

Except *abs(a)*, they all take an argument of type *double* and return a value of type *double*.

Note: The typical floating point types and its sizes in C++ are:

Type	size

float	4
double	8

4. PROGRAM INPUT USING THE `cin` OBJECT

- So far, our programs have been limited since that all their data must be defined within the program source code.
- We now learn how to write programs which enable data to be entered via the keyboard, while the program is running.
- **Standard Input Stream**

The *cin* object reads in information from the keyboard via the standard input stream.

The *extraction operator* (`>>`) retrieves information from the input stream.

When the statement `cin >> num1;` is encountered, the computer stops program execution and accepts data from the keyboard. When a data item is typed, the *cin* object stores the item into the variable listed after the `>>` operator.

Example 3.4.1

```
#include <iostream>
using namespace std;
int main()
{
    float num1, num2, product;
    cout << "Please type in a number: ";
    cin >> num1;
    cout << "Please type in another number: ";
    cin >> num2;
    product = num1 * num2;
    cout << num1 << " times " << num2 << " is " << product << endl;
    return 0;
}
```

The output of the above program:

Please type in a number: 30

Please type in another number: 0.05

30 times 0.05 is 1.5

Example 3.4.2

```
#include <iostream>
using namespace std;
int main()
{
    int num1, num2, num3;
    float average;

    cout << "Enter three integer numbers: ";
    cin >> num1 >> num2 >> num3;
    average = (num1 + num2 + num3) / 3.0;
    cout << "The average of the numbers is " << average << endl;
    return 0;
}
```

The output of the above program:

Enter three integer numbers: 22 56 73

The average of the numbers: 50.333333

5. Strings

- *Fundamental types* represent the most basic types handled by the machines where the code may run. But one of the major strengths of the C++ language is its rich set of *compound types*, of which the fundamental types are mere building blocks.

An example of compound type is the ***string*** class. Variables of this type are able to store sequences of characters, such as words or sentences. A very useful feature!

A first difference with fundamental data types is that in order to declare and use objects (variables) of this type, the program needs to include the header where the type is defined within the standard library (header `<string.h>`).

Position: 0 1 2 3 4

H	e	l	l	o
---	---	---	---	---

```
// my first string
#include <iostream>
#include <string>
using namespace std;
int main ()
{
    string mystring;
    mystring = "This is a string";
    cout << mystring;
    return 0;
}
```

- A variable of *string* type can be initialized in the variable declaration as follows:

```
    string month = "March";
```

- A variable of *string* type can be initialized in the second way as follows:

```
    string s1("Hello");
```

Example 3.5.1

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
    string str1; // an empty string
    string str2("Good Morning");
    string str3 = "Hot Dog";
    string str4(str3);
    string str5(str4, 4);
    string str6 = "linear";
    string str7(str6, 3, 3);
    cout << "str1 is: " << str1 << endl;
    cout << "str2 is: " << str2 << endl;
    cout << "str3 is: " << str3 << endl;
    cout << "str4 is: " << str4 << endl;
    cout << "str5 is: " << str5 << endl;
    cout << "str6 is: " << str6 << endl;
    cout << "str7 is: " << str7 << endl;
    return 0;
}
```

The output of the program:

```
str1 is:
str2 is: Good Morning
str3 is: Hot Dog
str4 is: Hot Dog
str5 is: Dog
str6 is: Linear
str7 is: ear
```

Some methods for string class

- `string objectName(str, n)` : creates and initializes a string object with a substring of string object *str*, starting at index position *n* of *str*.
- `string objectName(str, n, p)` : creates and initializes a string object with a substring of string object *str*, starting at index position *n* of *str* and containing *p* characters.

Input a string with `getline()`

- The expression `getline(cin, message)` will continuously accept and store characters typed at the terminal until the *Enter* key is pressed. All the characters encountered by `getline()` are stored in the string named *message*.

Example 3.5.2

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
    string message;
    cout << "Ener a string: \n");
    getline(cin, message);
    cout << "The string just entered is:\n") << message << endl;
    return 0;
}
```

The output of the program:

Enter a string:

This is a test input of a string

The string just entered is:

This is a test input of a string

← typed in by user